

CYBERSECURITY TOOLS · WEEKLY LEARNING TRACK

Nmap Reference Documentation

Timing Templates & Performance Tuning

Prepared by Emmanuel Aidoo

Kali Linux · Nmap 7.98

September 21, 2026

Contents

- 1 Minimum Rate Tuning (--min-rate)..... 3
- 2 Aggressive Timing via Zenmap (-T4) 4
- 3 Insane Timing (-T5)..... 5
- 4 Key Takeaways 6

Objective: To see how Nmap's speed and reliability trade off against each other.

Topic: forcing a minimum packet rate (--min-rate), stepping up through the built-in timing templates via Zenmap (-T4), and what happens to accuracy at the fastest setting (-T5). All scans below target 173.230.138.70.

1 Minimum Rate Tuning (--min-rate)

```
$ nmap --min-rate 1000 173.230.138.70
```

What I learned

--min-rate tells Nmap never to send probe packets slower than the given number per second, overriding its usual congestion-control pacing. It trades some of Nmap's automatic caution for raw speed. It is useful when you already trust the network can handle it.



Figure 1 • Forcing a 1000 packets/second minimum send rate against 173.230.138.70

Result summary

Port	State	Service
22/tcp	open	ssh
80/tcp	open	http
587/tcp	open	submission

My observation

997 of 1000 ports filtered, same three ports open as every earlier scan of this host but the whole thing finished in 7.10 seconds. That's close to the fastest result seen against this target across every scan so far, purely from forcing the packet rate rather than changing scan type.

2 Aggressive Timing via Zenmap (-T4)

```
$ nmap -sV -T4 -A -v 173.230.138.70
```

What I learned

Zenmap is Nmap's official graphical front-end with same scan engine, same output, just a GUI wrapped around the command line. This run used -T4 (“Aggressive”, one step below the fastest template) combined with -sV and -A for full version detection, OS attempts, scripts, and a traceroute.

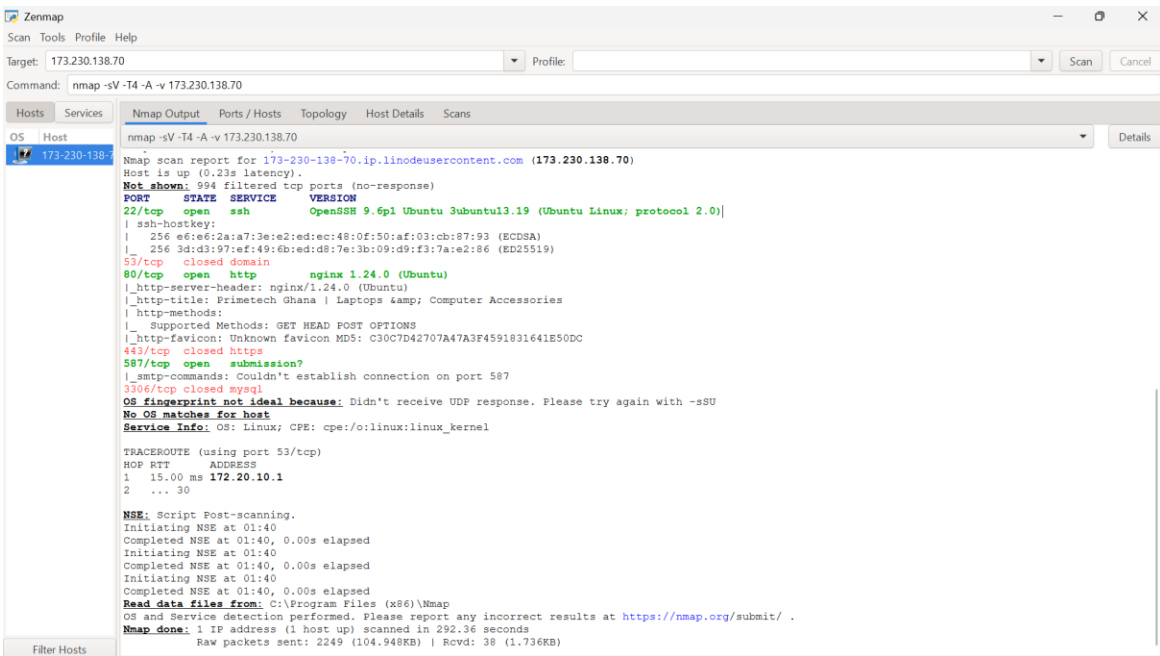


Figure 2 · Zenmap GUI running an intense -T4 scan with full version detection

Result summary

Port	State	Version detected
22/tcp	open	OpenSSH 9.6p1 Ubuntu 3ubuntu13.19 (protocol 2.0)
53/tcp	closed	—
80/tcp	open	nginx 1.24.0 (Ubuntu) — title: Primetech Ghana
443/tcp	closed	—

Port	State	Version detected
587/tcp	open	submission? (unconfirmed)
3306/tcp	closed	—

My observation

Every earlier scan of this host reported ports 22, 80, and 587 as tcpwrapped — open, but with no identifiable service. Here, the exact same ports resolved to real version strings: OpenSSH 9.6p1 and nginx 1.24.0. Nothing about the target changed; what's different is -sV was combined explicitly with -A this time, and Zenmap's default verbosity (-v) ran a fuller probe set. It's a reminder that tcpwrapped isn't necessarily permanent — a differently-configured scan against the same host can still pull a real banner where a previous one couldn't. Total scan time: 292.36 seconds, by far the longest in this module, reflecting how much more work -sV plus -A plus NSE scripting does per port versus a plain scan.

3 Insane Timing (-T5)

```
$ nmap -T5 -A 173.230.138.70
```

What I learned

-T5 is Nmap's fastest and least patient timing template — minimal delays between probes, aggressive parallelism, and short timeouts. It trades reliability for raw speed, and this scan shows exactly what that trade costs.

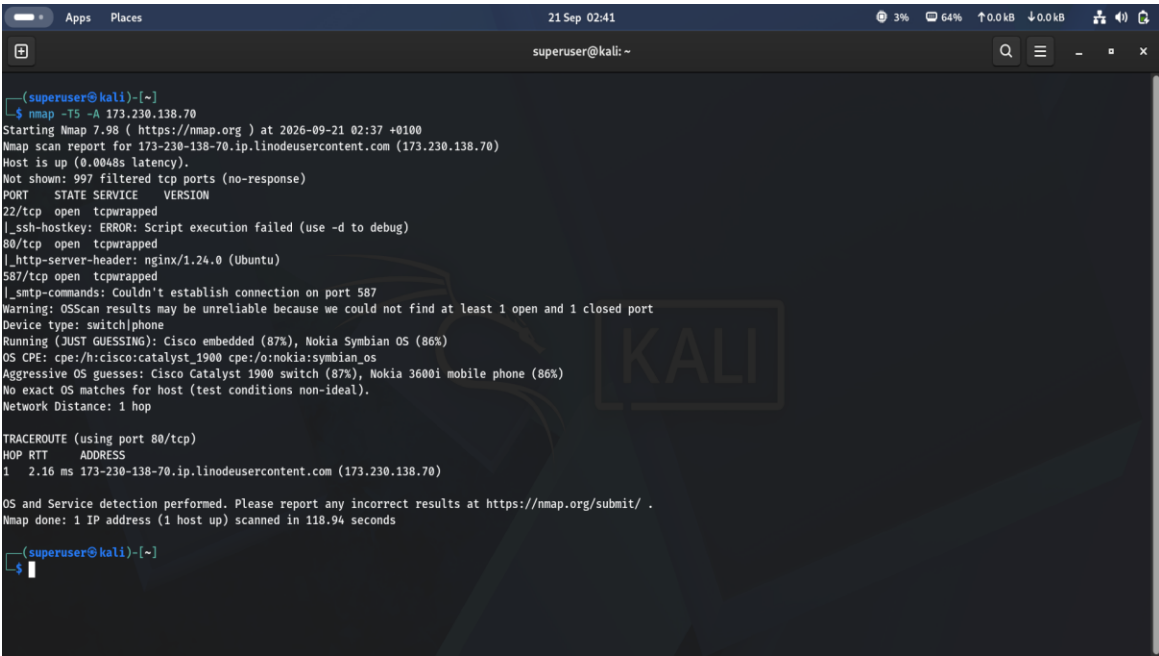


Figure 3 • Insane-speed -T5 scan degrading OS-detection accuracy

Result summary

Aspect	Result
Ports	Same 3 open (tcpwrapped) as prior -A runs — port scanning itself held up fine
OS detection	“Warning: OSScan results may be unreliable because we could not find at least 1 open and 1 closed port”
OS guesses	Device type: switch phone — “JUST GUESSING”: Cisco embedded (87%), Nokia Symbian OS (86%); aggressive guesses named a Cisco Catalyst 1900 switch and a Nokia 3600i mobile phone
Scan time	118.94 seconds — faster than the Zenmap run, but at a real accuracy cost

My observation

173.230.138.70 is a Linux server running nginx and OpenSSH, confirmed in Section 2. -T5's guesses a Cisco switch, a Nokia phone aren't just wrong, they're a completely different device class, and Nmap itself flagged the result as unreliable rather than presenting it as fact. This is the clearest illustration in the timing/accuracy trade-off: OS fingerprinting depends on precisely-timed responses, and -T5's aggressive pacing didn't leave room to collect them cleanly.

4 Key Takeaways

- --min-rate forces a floor on packet-send speed, letting you trade some of Nmap's built-in congestion caution for a faster scan when you trust the network.
- Zenmap is the same Nmap engine behind a GUI which is useful for building and re-running commands visually, not a different scanner.
- tcpwrapped is not a fixed verdict for a host the same port that stonewalled one scan configuration handed over a full version banner (OpenSSH, nginx) under a different one.
- Timing templates run from -T0 (paranoid) to -T5 (insane) higher isn't strictly better, since faster pacing can starve accuracy-dependent features like OS fingerprinting.
- When Nmap flags its own OS guess as unreliable, trust that warning over the guess itself wildly wrong device-class guesses (a Linux web server misread as a Cisco switch) are the direct result of too-aggressive timing, not a genuinely ambiguous host.